Foundations: What Makes an Agent

Prepare for technical interviews covering agentic AI

ASSUMES	General programming ability; no prior agent construction or framework experience (LangGraph, CrewAI, AutoGen, etc.)
AFTER THIS	Explain what distinguishes an agent from a plain LLM call or RAG pipeline, trace a single tool-use step (reason → act → observe), and describe the function calling API contract (schemas, parallel calls, strict JSON).

Agent Definition & Agency

An agent is a system that pursues a goal by repeatedly choosing actions, observing the results, and deciding what to do next. The defining move is the loop: reason → act → observe. A plain LLM call has no loop — it receives a prompt and returns a completion. A RAG pipeline has a fixed loop — retrieve, then generate — but the retrieval step is not chosen by the model; it is hard-coded into the pipeline.

CAPABILITY	PLAIN LLM CALL	RAG PIPELINE	AGENT
Goal pursuit	Single-turn response	Fixed retrieve-then-generate	Iterative, model-chosen steps
Action space	Text only	Text + fixed retrieval	Text + arbitrary tools (code, APIs, browser, etc.)
Control flow	None — one forward pass	Deterministic DAG	Dynamic, model-driven loop
Memory	Context window only	Context window + retrieved docs	Context window + external memory (scratchpad, vector store, episode log)
Failure recovery	None	None (pipeline fails or returns best effort)	Can retry, switch tools, re-plan

What distinguishes an agent from a plain LLM call and a RAG pipeline

The table makes the boundary sharp: an agent decides *whether* to act, *what* action to take, and *whether* the result satisfies the goal. In a RAG pipeline, the retrieval step is mandatory and its query is often constructed by a deterministic heuristic, not by the model's own reasoning. In an agent, the model itself emits a tool call — a structured decision — and the environment returns an observation that the model then conditions on.

1	**LLM Call**	Prompt → completion, no loop
2	**RAG Pipeline**	Fixed retrieve → generate, no model-chosen actions
3	**Single-Tool Agent**	Model chooses one tool per step, loops until done
4	**Multi-Tool Agent**	Model chooses among many tools, parallel calls, structured outputs
5	**Planning Agent**	Explicit plan generation, decomposition, re-planning on failure

Spectrum of agency: each step adds a capability the previous one lacks

Interviewers often probe the boundary with questions like: "If I wrap an LLM call in a while loop that retries on error, is that an agent?" The answer is no — retry logic is deterministic control flow, not model-chosen action. The model still does not decide *what* to do differently; the loop decides *whether* to repeat. Agency requires the model to author the next action conditioned on the previous observation.

> **A system uses an LLM to write a SQL query, executes it, and if the query fails, feeds the error back to the LLM to rewrite the query. Is this an agent? Why or why not?**
>
> → Yes. The model chooses the action (the SQL query), observes the result (success or error), and conditions its next action on that observation. The loop is model-driven, not a fixed retry.

Another common trap: "My RAG system uses an LLM to rewrite the user query before retrieval. Is that an agent?" Rewriting the query is still a fixed step in a predetermined pipeline. The model does not decide *whether* to retrieve, *what* to retrieve from, or *whether* the retrieved passages are sufficient. It only reformulates. That is a prompt engineering technique, not agency.

reason → act → observe loop | model-chosen actions | dynamic control flow | external memory | failure recovery

LLM as Reasoning Engine

A plain LLM call is a completion engine: you give it a prompt, it predicts the next tokens. An agent treats the LLM as a reasoning engine: you give it a goal, it plans steps, decides which tool to invoke, observes the result, and reflects on whether to continue or change course. The difference is not the model — it is the control loop that wraps the model.

Completion engine	Single prompt → single response, no loop
Reasoning engine	Goal → (plan → act → observe → reflect)* → answer

Completion vs. reasoning: two ways to use the same model

In a RAG pipeline the LLM is still a completion engine: retrieve context, stuff it into a prompt, get an answer. There is no decision about *whether* to retrieve, *what* to retrieve, or *whether* the answer is sufficient. An agent adds those decisions. The LLM emits structured decisions ("call search with query X") instead of free text, and the runtime feeds the tool result back for the next reasoning step.

WORKED EXAMPLE

Single tool-use step traced

1. Goal: "What is the current temperature in Tokyo?"
2. Reason (LLM): decides it needs live data → emits function call `get_weather({"city": "Tokyo"})`
3. Act (runtime): executes the function, receives `{"temp_c": 18, "condition": "cloudy"}`
4. Observe (LLM): receives the tool result as a new message
5. Reflect (LLM): decides the answer is complete → emits final response "It's 18 °C and cloudy in Tokyo."

> **A developer wraps an LLM call in a `while` loop that re-prompts the model until it says "DONE". Is this an agent?**
>
> → No. The loop is hard-coded; the LLM never decides *whether* to continue or *what* to do next. An agent requires the LLM to make those decisions (plan, act, observe, reflect) on each iteration.

Function Calling / Tool Calling API

Function calling is the primitive that turns a language model into an agent. Without it, the model can only emit text. With it, the model emits a structured decision — a name and arguments — that your code executes, then feeds the result back. The loop is: reason (choose a tool), act (call it), observe (read the result), repeat. Every agent framework (LangGraph, CrewAI, AutoGen) builds on this same contract.

The API Contract

The model receives a list of tool definitions. Each definition is a JSON Schema describing the function name, description, and parameters. The model responds with a `tool_calls` array (OpenAI) or `tool_use` blocks (Anthropic). Your code executes each call and returns a `tool` message with the result. The model then continues.

```
// OpenAI tool definition
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather for a location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "City and state, e.g. 'San Francisco, CA'"
        },
        "unit": {
          "type": "string",
          "enum": ["celsius", "fahrenheit"],
          "description": "Temperature unit"
        }
      },
      "required": ["location"],
      "additionalProperties": false
    },
    "strict": true
  }
}
```

Tool definition with strict JSON Schema. `strict: true` guarantees the model outputs valid JSON matching the schema.

```
// Model response with tool_calls
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": null,
      "tool_calls": [{
        "id": "call_abc123",
        "type": "function",
        "function": {
          "name": "get_weather",
          "arguments": "{\"location\": \"San Francisco, CA\", \"unit\": \"celsius\"}"
        }
      }]
    }
  }]}
```

Model emits a tool call. `arguments` is a JSON string — parse it, don't eval.

```
// Your code returns the tool result
{
  "role": "tool",
  "tool_call_id": "call_abc123",
  "content": "{\"temperature\": 18, \"condition\": \"foggy\", \"unit\": \"celsius\"}"
}
```

Tool result message. `tool_call_id` must match the call. Content is a string (JSON or plain text).

KEY SCHEMA RULES

- Set `additionalProperties: false` on the root parameters object — prevents hallucinated keys.
- Mark all required fields in `required` array — the model will omit optional ones.
- Use `enum` for closed sets (units, categories, status codes).
- Descriptions are prompts — write them for the model, not for humans.
- Keep schemas flat; nested objects increase error rate.

Parallel Calls

The model can emit multiple `tool_calls` in one response. Your executor must run them concurrently (or sequentially, but parallel is faster) and return all results before the next model turn. This is how an agent fetches weather for three cities in one round trip.

```
import asyncio
import json
from openai import AsyncOpenAI

client = AsyncOpenAI()

async def run_parallel_tool_calls():
    tools = [{
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather for a location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {"type": "string"},
                    "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
                },
                "required": ["location"],
                "additionalProperties": false
            },
            "strict": true
        }
    }]

    resp = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": "Weather in NYC, London, Tokyo in celsius"}],
        tools=tools,
        tool_choice="auto"
    )

    tool_calls = resp.choices[0].message.tool_calls
    if not tool_calls:
        return resp.choices[0].message.content

    # Execute all calls in parallel
    async def execute(call):
        args = json.loads(call.function.arguments)
        # In reality, call your actual API here
        result = await fake_weather_api(args["location"], args.get("unit", "celsius"))
        return {
            "role": "tool",
            "tool_call_id": call.id,
            "content": json.dumps(result)
        }

    tool_results = await asyncio.gather(*[execute(c) for c in tool_calls])

    # Second turn: feed results back
    final = await client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "user", "content": "Weather in NYC, London, Tokyo in celsius"},
            resp.choices[0].message,
```

```
            *tool_results
        ],
        tools=tools
    )
    return final.choices[0].message.content

async def fake_weather_api(location, unit):
    # Simulated external call
    await asyncio.sleep(0.1)
    return {"location": location, "temperature": 20, "unit": unit}

print(asyncio.run(run_parallel_tool_calls()))
```

Parallel tool execution pattern. `asyncio.gather` runs all calls concurrently. The second turn includes the assistant's tool_calls message AND all tool results.

Strict JSON Mode

Set `"strict": true` in the function definition. This constrains the model's token generation to only produce valid JSON matching your schema. Without it, the model may emit trailing commas, unquoted keys, or hallucinated fields. Strict mode adds latency (constrained decoding) but eliminates a whole class of parsing errors. Use it in production.

MODE	VALID JSON GUARANTEED	LATENCY	USE CASE
strict: false	No	Lower	Prototyping, trusted models
strict: true	Yes (constrained decoding)	Higher	Production, untrusted inputs, any schema with enums/required
response_format: {type: 'json_object'}	Yes (but no schema validation)	Medium	General JSON output, not tool calls

JSON mode trade-offs. Strict mode is the only one that validates against your schema during generation.

Tracing a Single Tool-Use Step

Interviewers ask you to trace: user question → model reasoning → tool call → execution → result → model response. Here is the complete trace for "What's the weather in Boston?"

WORKED EXAMPLE

Trace: Single Tool-Use Step (Reason → Act → Observe)

1. USER: "What's the weather in Boston?"
2. MODEL (reasoning): User wants current weather. I have get_weather tool. Need location="Boston, MA", unit default celsius.
3. MODEL (tool_calls): [{"id": "call_1", "name": "get_weather", "arguments": "{\"location\": \"Boston, MA\"}"}]
4. YOUR CODE (act): Parse arguments → call weather API with location="Boston, MA" → get {"temp": 15, "condition": "rainy"}
5. YOUR CODE (tool message): {"role": "tool", "tool_call_id": "call_1", "content": "{\"temp\": 15, \"condition\": \"rainy\"}"}
6. MODEL (observe + respond): Sees tool result. Generates final answer: "It's 15°C and rainy in Boston."

1	**User Query**	Natural language request
2	**Model Reasoning**	Selects tool, fills arguments
3	**Tool Call Emitted**	Structured JSON with id, name, arguments
4	**Executor Runs Tool**	Your code: parse, call API, catch errors
5	**Tool Result Returned**	Tool message with matching tool_call_id
6	**Model Observes & Responds**	Incorporates result into final answer

The reason → act → observe loop. Each step precedes the next. The executor (your code) owns steps 3–4.

Common Interview Traps

- "What happens if the model hallucinates a tool name?" → You get a tool_calls entry for a function you didn't define. Return an error tool message: {"role": "tool", "tool_call_id": id, "content": "Error: unknown function"}. The model then recovers.
- "What if arguments fail schema validation?" → With strict: true, this cannot happen. Without it, catch JSON parse errors and return a tool error message.
- "How do you handle a tool that takes 30 seconds?" → Set a timeout. On timeout, return a tool message with error content. The model sees the failure and can retry or apologize.
- "Can the model call a tool, see the result, then call another?" → Yes. That's multi-turn tool use. Each turn: model emits calls → you execute → you return results → model decides next step.

> **The model emits a tool call with arguments {"location": "Paris", "unit": "kelvin"} but your schema only allows enum ["celsius", "fahrenheit"]. What happens with strict: true vs strict: false?**
>
> → strict: true — the model physically cannot emit "kelvin"; constrained decoding blocks invalid tokens. strict: false — the model emits it, your JSON parser accepts it, your executor must validate and return an error tool message.

> **In a parallel call scenario, the model emits 3 tool_calls. Your second call fails (API timeout). How do you structure the tool messages for the next turn?**
>
> → Return 3 tool messages with the same tool_call_ids. For the failed one, content is an error string like "Error: timeout". The model sees all three results simultaneously and decides how to proceed.

Structured Output / JSON Mode

An agent is a loop: the model reasons, emits a structured action, the environment executes it, and the model observes the result. If the model emits free-form text, the downstream parser must guess where the action starts, what its name is, and what arguments it carries. That guesswork breaks autonomy — the loop stalls or executes the wrong thing. Structured output removes the guesswork by constraining the model to emit only valid JSON that conforms to a declared schema.

There are two mechanisms. JSON mode (sometimes called response_format) tells the model: "Your entire reply must be a single JSON object." It gives you syntactic validity but no semantic contract — the model can invent any keys it likes. Function calling (tool calling) goes further: you supply a JSON Schema for each tool, and the model must pick one tool (or several in parallel) and emit arguments that validate against that exact schema. The API returns a tool_calls array with name, id, and arguments already parsed. Frameworks such as LangGraph, CrewAI, and AutoGen build their entire orchestration on this contract: they register tools as schemas, invoke the model, receive validated tool_calls, execute them, and feed the results back as tool messages.

```
// OpenAI function-calling request (simplified)
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "You have access to tools."},
    {"role": "user", "content": "What's the weather in Tokyo?"}
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get current weather for a location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": {"type": "string", "description": "City name"},
            "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
          },
          "required": ["location"],
          "additionalProperties": false
        },
        "strict": true
      }
    }
  ],
  "tool_choice": "auto"
}
```

Function-calling request: schema + strict=true guarantees the model emits only valid arguments.

```
// Model response with tool_calls
{
  "choices": [{
    "message": {
      "role": "assistant",
      "tool_calls": [{
        "id": "call_abc123",
        "type": "function",
        "function": {
          "name": "get_weather",
          "arguments": "{\"location\": \"Tokyo\", \"unit\": \"celsius\"}"
        }
      }]
    }
  }]
}
```

Model emits a tool_call with validated arguments (JSON string). The framework parses and executes.

10

PROPERTY	JSON MODE (RESPONSE_FORMAT)	FUNCTION CALLING (TOOL_CALLS)
Output shape	Single JSON object	Array of tool_calls (each with name, id, arguments)
Schema enforcement	None (model invents keys)	Strict: arguments must match declared JSON Schema
Parallel calls	Not supported	Native: model can emit multiple tool_calls in one turn
Strict JSON (no stray commas/comments)	Best effort	Guaranteed when strict=true
Use case	One-off structured extraction	Agent tool use, multi-step loops

JSON mode vs function calling — what each guarantees

The strict flag (strict: true in the schema) is the linchpin. Without it, the model may emit trailing commas, comments, or keys not in the schema — valid JSON but invalid against your contract. With strict=true, the API pre-compiles the schema into a constrained grammar and forces the tokenizer to follow it. You get parseable output every time, which is what lets an agent loop run unattended.

> **A candidate says: "I'll just use JSON mode and ask the model to output {\"tool\": \"get_weather\", \"args\": {...}} — same thing, right?" Why is this wrong for an agent loop?**
>
> → JSON mode has no schema enforcement — the model can rename keys, omit required fields, or add commentary. Function calling validates arguments against the declared schema at generation time (strict=true) and returns pre-parsed tool_calls. The framework cannot reliably dispatch a tool from free-form JSON.

Tool Use / Function Calling Pattern

An agent differs from a plain LLM call or RAG pipeline in one structural way: it can *act* on the world through tools, observe the result, and decide what to do next. A plain LLM call produces text and stops. A RAG pipeline retrieves context, stuffs it into the prompt, and produces text. An agent closes the loop: reason → act → observe → reason again.

The Single Tool-Use Step: Reason → Act → Observe

Every agent framework (LangGraph, CrewAI, AutoGen, OpenAI Assistants, Anthropic tool use) implements the same base pattern. One step consists of three phases:

1. **Reason** — The model receives the conversation history (system prompt, user messages, prior tool results) and decides whether to call a tool, and if so, which one and with what arguments. 2. **Act** — The framework executes the tool with those arguments. This is *your* code: a Python function, an HTTP request, a database query. 3. **Observe** — The tool's return value (or error) is formatted as a tool-result message and appended to the conversation. The model now sees the outcome and reasons again.

WORKED EXAMPLE

Trace: User asks "What's the weather in Tokyo?"

1. [1] USER MESSAGE → conversation: [{"role": "user", "content": "What's the weather in Tokyo?"}]
2. [2] REASON → model sees conversation + tool schemas. It decides to call get_weather({"city": "Tokyo"}). Output: {"tool_calls": [{"id": "call_abc123", "name": "get_weather", "arguments": "{\"city\": \"Tokyo\"}"}]}
3. [3] ACT → framework parses tool_calls, invokes your Python function get_weather(city="Tokyo"). Your code calls an API, returns {"temp_c": 18, "condition": "cloudy"}.
4. [4] OBSERVE → framework appends tool result message: {"role": "tool", "tool_call_id": "call_abc123", "name": "get_weather", "content": "{\"temp_c\": 18, \"condition\": \"cloudy\"}"}
5. [5] REASON (again) → model now sees the tool result. It produces final answer: "It's 18°C and cloudy in Tokyo."
6. [6] FINAL → conversation ends with assistant message: {"role": "assistant", "content": "It's 18°C and cloudy in Tokyo."}

Function Calling API Contract

The contract between model and framework is defined by JSON Schema. You provide a list of tool *schemas*; the model promises to emit arguments that validate against those schemas. Three details matter for interviews:

ASPECT	WHAT THE MODEL SEES	WHAT YOU MUST HANDLE
Schema	JSON Schema (Draft 2020-12 subset). Properties = parameters. Required array = mandatory args.	Define schemas precisely. Extra properties are rejected if `additionalProperties: false`.
Parallel calls	Model may emit multiple tool_calls in one response (e.g., get_weather for Tokyo AND Seoul).	Your framework must dispatch all calls, collect all results, append *all* tool messages before the next model turn.
Strict JSON / `strict: true`	OpenAI's `strict: true` forces the model to output only valid JSON matching the schema — no prose, no markdown fences.	If you don't use strict mode, you must parse defensively: strip markdown, handle trailing commas, recover from partial JSON.
Tool result format	A message with `role: "tool"`, `tool_call_id` (must match the call), `name`, and string `content`.	Serialize your return value to JSON string. Errors: return error info in content; the model decides how to proceed.
System prompt role	Describes *when* to use each tool, not *how*. The schema describes *how*.	Keep system prompt focused on policy (e.g., "always verify user location before weather").

Function calling contract essentials

Minimal Working Code (No Framework)

You can implement the loop in ~30 lines. This is what every framework wraps:

```
import json
from openai import OpenAI

client = OpenAI()

def get_weather(city: str) -> dict:
    # Your real implementation here
    return {"temp_c": 18, "condition": "cloudy"}

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
            "additionalProperties": False
        },
        "strict": True
    }
}]

messages = [
    {"role": "system", "content": "You have access to get_weather. Use it when users ask for
weather."},
    {"role": "user", "content": "What's the weather in Tokyo?"}
]

# 1. REASON
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    tools=tools,
    tool_choice="auto"
)
msg = response.choices[0].message
messages.append(msg.model_dump())

# 2. ACT (if tool calls exist)
if msg.tool_calls:
    for tc in msg.tool_calls:
        args = json.loads(tc.function.arguments)
        result = get_weather(**args)
        # 3. OBSERVE
        messages.append({
            "role": "tool",
            "tool_call_id": tc.id,
            "name": tc.function.name,
            "content": json.dumps(result)
        })
    # 4. REASON again → final answer
    final = client.chat.completions.create(
        model="gpt-4o-mini",
```

```
        messages=messages,
        tools=tools
    )
    print(final.choices[0].message.content)  # "It's 18°C and cloudy in Tokyo."
```

Single-step tool-use loop (OpenAI-compatible)

> **The model returns a tool_calls array with two entries: get_weather({"city": "Tokyo"}) and get_weather({"city": "Seoul"). Your code executes both. How many tool-result messages must you append before the next model call?**
>
> → Two. Each tool_call has a unique `tool_call_id`. You must append one tool-result message per call, each carrying its matching `tool_call_id`. The model expects to see *all* results before it reasons again.

> Interviewers often ask: "What happens if the model calls a tool that doesn't exist?" Answer: The framework should catch the unknown tool name, return an error tool-result (e.g., `{"error": "unknown tool: foo"}`), and let the model recover. Never crash the loop.

Generated 2026-09-26 · template study-notes@31f7aa7febc1